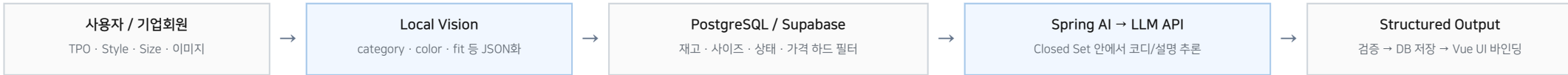


FITLY AI 아키텍처 및 운영 전략 브리핑

2026.09.03
FITLY AI / Backend 설계

왜 하이브리드 AI(Local Vision + API LLM)인가 · 운영 · 비용 · 배포 · 확장성

설계 한 줄 요약 · FITLY는 “이미지를 이해하는 일”과 “코드를 판단하고 설명하는 일”을 분리한다. 의류 이미지는 Local Vision으로 1회 구조화하여 재사용하고, 재고·사이즈 같은 하드 조건은 PostgreSQL이 먼저 걸러낸 뒤, Spring AI를 통해 외부 LLM API가 제한된 후보 안에서 스타일링 Reasoning만 수행한다. 이는 AI를 많이 쓰기 위한 설계가 아니라 비용·무결성·운영 복잡도를 최소화하면서 필요한 곳에만 AI를 쓰기 위한 설계다.



1. 왜 이렇게 설계했는가?

FITLY의 문제는 단순한 “비슷한 옷 검색”이 아니다. 사용자의 비정형 의류 사진을 데이터화하고, 실제 창고의 사이즈·재고·가격을 만족하면서 TPO와 스타일이 어울리는 조합을 만들어야 한다.

Vision: 이미지 픽셀을 category/color/fit 같은 구조화 데이터로 변환.

DB: available_stock > 0, size, status, price 등 결정적인 조건을 보장.

LLM: DB가 할 수 없는 색상 조화, 실루엣 균형, TPO 적합성, 설명 생성을 담당.

추천 결과는 자유문장이 아니라 JSON으로 받아 RECOMMENDATION / RECOMMENDATION_ITEM에 연결한다.

핵심 원칙: DB = 사실의 원천(Source of Truth), AI = 제한된 범위의 판단 도구.

3. 운영 전략: Hallucination과 데이터 무결성 통제

RAG/Vector DB를 기본 추천 경로에서 사용하지 않음: 렌탈의 핵심은 의미 유사도보다 실시간 재고와 물리적 조건이다.

DB에서 먼저 대여 가능한 상품 3~5개를 추린 뒤 LLM에 Closed Set Candidate로 전달.

LLM이 반환한 productId는 서버에서 후보군에 실제 존재하는지 재검증.

가격·재고·총 결제액은 LLM 결과를 신뢰하지 않고 DB snapshot으로 다시 계산.

Vision 인식 오류는 UI에서 수정 가능. Enterprise 설명도 AI 초안 → MD 검토 → 최종 등록.

AI timeout/JSON 파싱 실패는 제한적 retry 후 FAILED 처리; 무한 retry 금지.

추천 점수는 UI용 상대 점수로 사용하고, DB의 matching_score가 0~1이면 85점 → 0.8500으로 정규화해 저장한다.

5. 비용 전략: AI 비용을 기능 단위로 통제

Vision 비용	LLM 비용	DB/Storage	생성 이미지
이미지 등록 시 1회 Local 추론 → 결과 캐시. 추천마다 재분석하지 않음.	전체 상품이 아니라 후보 3~5개 JSON만 전송. 짧은 Structured Output 사용.	텍스트·메타데이터는 PostgreSQL, 무거운 이미지는 파일 스토리지로 분리.	착용 이미지는 사용자가 눌렀을 때만 생성. 장기 저장하지 않고 필요 시 TTL 삭제.

초기에는 Supabase의 무료/저비용 구간과 표준 PostgreSQL을 활용해 별도 NoSQL/Vector DB 비용을 만들지 않는다.

Prompt/Schema는 짧고 고정적으로 유지하고, output token 상한·timeout·retry 횟수를 제한한다.

운영 지표: cost/request input/output tokens AI latency failure rate vision reuse rate

2. 왜 Local Vision + API LLM의 혼합 구조인가?

방식	장점	문제
전부 API	구현이 빠름	같은 이미지 반복 전송, 이미지 입력 비용, 개인정보 외부 전달, 결과 변동
전부 Local	호출비 절감	대형 LLM GPU 상시 운영, 모델 업데이트·장애·서빙 관리 부담
하이브리드 (채택)	각 모델의 강점만 사용	명확한 JSON 계약과 모델 경계 관리 필요

Vision을 Local로 두는 이유: 의류의 반복 가능한 일반 특징은 등록 시 한 번 분석해 저장하면 된다. 외부 Vision API를 매번 호출하지 않아 비용을 줄이고, 원본 이미지의 외부 전송을 최소화하며, 인식값을 사용자가 직접 수정하는 Human-in-the-loop도 쉽게 적용할 수 있다.

LLM을 API로 두는 이유: 코드 Reasoning과 자연어 생성은 대형 모델의 품질이 중요하지만, 이를 로컬에서 24시간 운영하면 GPU·서빙·모델 관리 비용이 커진다. MVP에서는 Spring AI로 사용자 기반 API를 호출하는 편이 운영상 합리적이다.

4. 배포 전략: 지금은 단순하게, 확장 시 분리

데이터베이스 설계와 동일하게 초기 단계에서는 서버 수를 늘리는 것보다 운영 단순성을 우선한다.

운영 DB: Supabase PostgreSQL. 로컬 개발 DB와 운영 데이터를 분리해 개발 실수와 테스트 트래픽을 격리.

Core App: Vue + Spring Boot + Spring AI 중심의 단일 애플리케이션 배포를 유지.

Vision: 업로드 시점의 경량 로컬 추론으로 핵심 트랜잭션 경로와 분리. 결과 JSON을 저장하여 반복 추론 제거.

향후 트래픽/모델이 커지면 Vision Provider 계약은 유지한 채 전용 GPU 서버/FastAPI 등의 독립 서비스로 이전 가능.

대규모 동시 주문은 현재 DB transaction/lock으로 처리하고, 필요 시 Kafka queue로 확장.

배포 원칙: 현재의 단순한 구조를 깨뜨리지 않고, 부하가 실제로 발생한 지점만 나중에 분리한다.

6. 최종 판단: FITLY가 얻는 효과

정확성: 존재하는 재고와 variant만 추천하므로 실제 대여/결제로 연결 가능.

비용: 반복 이미지 API 호출과 대형 로컬 LLM 운영을 동시에 피함.

운영성: Spring AI로 LLM Provider를 추상화하고, Mock Provider로 CI에서 외부 API 비용 없이 테스트.

개인정보: 의류 원본 이미지는 가능한 한 로컬에서 처리하고 LLM에는 구조화 JSON 위주로 전달.

확장성: PostgreSQL 표준 구조와 Provider 경계를 유지해 향후 GPU 서비스·Kafka·온프레미스로 단계적 확장 가능.

결론: “전부 AI”가 아니라 Local Vision → DB hard filtering → API LLM reasoning → 서버 검증으로 책임을 나눠야 실제 서비스의 비용·안정성·설명가능성을 동시에 확보할 수 있다.